

Week 8 feed forward neural network and backpropagation notes

1. What is feed forward neural network?

FFNN is a neural network where information flows in one direction, from input through hidden layers to output. Each node in one layer is connected to every node in the next layer.

It contains an input layer, one or more hidden layers and an output layer. There are connections from nodes in one layer to nodes in the next. These are marked with weights.

2. How does going forward work in FFNN?

There are one node for each feature, plus the bias. We take the sum of the values multiplied with weights (wx). The result is run through an activation function $g(z)$ before adding the bias term and being fed to next layer.

3. What is the activation function?

Should be differentiable. For example sigmoid function $1/1+e^{-z}$. derivative is $y(1-y)$

4. What does the output layer look like in regression, binary classification, multi-label classification, and multi-class classification?

Regression: one output node. No activation function in output layer

Binary classification: one output layer. Logistic activation function in output layer

Multi-label classification: several output nodes. Logistic activation function

Multi-class classification: several output nodes. Sum the weighted inputs at each nodes. The sums are brought together in the soft-max

5. What are the steps in training?

Given a set of training instances $\{(x_1, t_1), (x_2, t_2), (x_3, t_3) \dots (x_n, t_n)\}$

Run the input forward: weighted sum($x*w$) and add activation in hidden layer. Add bias and send them forward to next layer. repeat until reaching output layer.

Output layer: depends but sum weighted unput sum(xw)

Use loss function and compare to target value. Apply gradient descent to update the weights.

6. What is backpropagation?

Backpropagation is a way to train the classifier by updating the weights at each layer.

the algorithm uses gradient descent optimization to adjust the weights of the connections between the neurons in the network, in order to minimize the error between the network's predicted output and the desired output.

During the training process, the inputs are passed through the network and the output is computed. The difference between the predicted output and the desired output is then computed, and this difference is called the error. The backpropagation algorithm then works by propagating this error backwards through the network, layer by layer, and computing the gradient of the error with respect to the weights of each connection. The weights are then adjusted in the opposite direction of the gradient, with the goal of minimizing the error.

7. How does the forwards work?

Add bias (x_0) and send through the first layer to get

$$H = xV = (h_1, h_2, \dots, h_k)$$

$$h_j = \text{sum}(xv)$$

k is the number of hidden nodes. Apply activation function to get

$$a = g(h) = (a_1, a_2, \dots, a_k)$$

$$\text{where } a_j = g(h_j)$$

add bias (a_0) and send through second layer to get

$$z = aW = (z_1, z_2, \dots, z_n)$$

$$z_j = \text{sum}(aW)$$

n is number of output nodes. If needed, apply activation function to get $y = f(z)$

8. How do backpropagation work in (linear) regression?

Only one output node with no activation function in output. We use SE

$$\text{Sum of Squares Error (SE): } L_{SE}(\mathbf{t}, \mathbf{y}) = \frac{1}{2} \sum_{j=1}^N (t_j - y_j)^2$$

We find the derivative of this loss with respect to W which is derivative of loss* derivative of a with respect to w. $(y-t)(a_i)$

$$\bullet \frac{\partial}{\partial w_{i,1}} L_{SE}(\mathbf{t}, \mathbf{y}) = \frac{\partial}{\partial y} L_{SE}(\mathbf{t}, \mathbf{y}) \left(\frac{\partial}{\partial w_{i,1}} y \right) = (t - y)(-a_i) = (y - t)(a_i)$$

a is output for last hidden layer.

$$\frac{\partial}{\partial w_{i,1}} L_{SE}(\mathbf{t}, \mathbf{y}) = (y - t)(a_i)$$

So this is how we update the last layer near the output.

To update the first layer, we have to take the derivative of loss with respect to V

$$\frac{\partial}{\partial v_{i,j}} L_{SE}(t, y) =$$

$$\frac{\partial}{\partial a_j} L_{SE}(t, y) = \frac{\partial}{\partial y} L_{SE}(t, y) \left(\frac{\partial}{\partial a_j} y \right) = (t - y)(-w_{j,1}) = (y - t)(w_{j,1}) =$$

$$\left(\frac{\partial}{\partial v_{i,j}} a_j \right) = \left(\frac{\partial}{\partial h} g \right) \left(\frac{\partial}{\partial v_{i,j}} h \right) =$$

- $\frac{\partial}{\partial v_{i,j}} h_j = x_i$
- If $a_j = g(h_j) = \sigma(h_j)$, then
 - $\left(\frac{\partial}{\partial h_j} g \right) = a_j(1 - a_j)$
 - $\left(\frac{\partial}{\partial v_{i,j}} a_j \right) = a_j(1 - a_j)x_i$

Put it all together $d(L) \cdot d(y) \cdot d(a) = d(L) \cdot d(y) \cdot d(g) \cdot d(h) =$

$$\delta_o(\kappa) (y - t)(w_{j,1}) a_j (1 - a_j) x_i$$

- $w_{i,1} = w_{i,1} - \eta \delta_o(\kappa_1) a_i$
- $v_{i,j} = v_{i,j} - \eta \delta(\text{hidden}_j) x_i$

$$\delta(\text{hidden}_j) = \delta_o(\kappa_1) (w_{j,1}) a_j (1 - a_j)$$

$$\delta_o(\kappa_1) = (y - t)$$

9. What is delta term?

(y-t)

10. How do the backpropagation work in binary classification?

Same as regression, but since it add an activation function in output the delta is different. Rest is same

$$\delta_o(\kappa_1) = (y - t)y(1 - y)$$

This is for square error loss. If we instead use cross entropy loss then

$$\delta_o(\kappa_1) = -\frac{(t-y)}{y(1-y)}y(1-y) = (y-t)$$

11. How do the backpropagation work in multi-label classification classification?

This has several output nnides and logistic activation function in output.

$$\delta_o(\kappa_j) = (y_j - t_j)y_j(1 - y_j)$$

- $\delta(hidden_j) =$
- $a_j(1 - a_j) \sum_{i=1}^n \delta_o(\kappa_i)w_{j,i}$

12. How do the backpropagation work in mutli-class classification?

- $\delta_o(\kappa_j) = (y_j - t_j)$
- As before:
 - $\delta(hidden_j) =$
 - $a_j(1 - a_j) \sum_{i=1}^n \delta_o(\kappa_i)w_{j,i}$
- Update the weights:
 - $w_{ij} = w_{i,j} - \eta \delta_o(\kappa_j)a_{i,j}$
 - $v_{i,j} = v_{i,j} - \eta \delta(hidden_j)x_i$

13. How can we avoid overfitting?

We can avoid overfitting by stopping earlier. If we have a validation set that we test while we train, we can stop the training when the loss in validation starts to increase

14. What other activation functions can we use besides sigmoid?

We have tanh and RELU.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$ReLU(x) = \max(x, 0)$$

15. How can we calculate accuracy, precision and recall?

Accuracy is (tn+tp)/N

Precision is tp/(tp+fp)

Recall is tp/(tp+fn)

16. What is F-score?

F-score combine precision and recall. It punishes smaller numbers more.

$$F_1 = \frac{2PR}{P+R} \left(= \frac{1}{\frac{\frac{1}{R} + \frac{1}{P}}{2}} \right)$$